

# Simple Linear Regression from Scratch to Scikit-learn

**A beginner-friendly, end-to-end walkthrough of Simple Linear Regression** — one of the foundational supervised learning algorithms in Machine Learning.

This notebook covers:

1. What Linear Regression is and the math behind it
2. Building a synthetic dataset (Years of Experience vs Salary)
3. Exploratory Data Analysis (EDA)
4. Training a model with scikit-learn
5. Evaluating performance (MAE, MSE, RMSE, R<sup>2</sup>)
6. Visualizing the regression line and residuals
7. Making predictions on new data

---

Author: Sambit Kumar Sahu | [GitHub: Sambit7848](#)

## 1. What is Linear Regression?

Linear Regression models the relationship between an independent variable **X** and a dependent variable **y** by fitting a straight line:

$$y = b_0 + b_1 \cdot X + \text{error}$$

- **b<sub>0</sub>** — the intercept (predicted value of y when X = 0)
- **b<sub>1</sub>** — the slope/coefficient (how much y changes for a unit change in X)
- **error** — the difference between the actual and predicted values

The model learns **b<sub>0</sub>** and **b<sub>1</sub>** by minimizing the **Sum of Squared Errors (SSE)** between predicted and actual values — this is called **Ordinary Least Squares (OLS)**.

## 2. Import Libraries

```
In [1]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score

plt.rcParams["figure.figsize"] = (7, 4.5)
np.random.seed(42)
```

## 3. Create the Dataset

For this example we simulate a realistic **Years of Experience vs Salary** dataset — a classic, intuitive use case for simple linear regression.

```
In [2]: n = 60
years_experience = np.round(np.random.uniform(0, 12, n), 1)
salary = 30000 + 8500 * years_experience + np.random.normal(0, 9000, n)

df = pd.DataFrame({
    "YearsExperience": years_experience,
    "Salary": np.round(salary, 2)
}).sort_values("YearsExperience").reset_index(drop=True)

df.head(10)
```

```
Out[2]:
```

|   | YearsExperience | Salary   |
|---|-----------------|----------|
| 0 | 0.2             | 40979.00 |
| 1 | 0.4             | 41638.62 |
| 2 | 0.5             | 31165.57 |
| 3 | 0.6             | 49181.79 |
| 4 | 0.7             | 38866.76 |
| 5 | 0.8             | 37583.42 |
| 6 | 1.1             | 26611.66 |
| 7 | 1.2             | 43414.01 |
| 8 | 1.5             | 35473.56 |
| 9 | 1.7             | 56656.16 |

## Quick statistical summary

```
In [3]: df.describe().round(2)
```

```
Out[3]:
```

|       | YearsExperience | Salary    |
|-------|-----------------|-----------|
| count | 60.00           | 60.00     |
| mean  | 5.61            | 76946.34  |
| std   | 3.66            | 31140.30  |
| min   | 0.20            | 26611.66  |
| 25%   | 2.20            | 50997.26  |
| 50%   | 5.40            | 73009.88  |
| 75%   | 8.57            | 101190.43 |
| max   | 11.60           | 136981.52 |

## 4. Exploratory Data Analysis

A quick scatter plot helps confirm a roughly linear relationship before modeling.

```
In [4]: plt.scatter(df["YearsExperience"], df["Salary"], color="#2563eb", alpha=0.7)
plt.xlabel("Years of Experience")
plt.ylabel("Salary ($)")
plt.title("Experience vs Salary — Raw Data")
plt.grid(alpha=0.25)
plt.show()
```



## 5. Train/Test Split & Model Training

We hold out 20% of the data for testing so we can evaluate the model on data it hasn't seen.

```
In [5]: X = df[["YearsExperience"]].values
```

```

y = df["Salary"].values

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model = LinearRegression()
model.fit(X_train, y_train)

print(f"Intercept (b0): {model.intercept_:.2f}")
print(f"Coefficient (b1): {model.coef_[0]:.2f}")

```

Intercept (b0): 29826.21  
Coefficient (b1): 8351.93

**Interpretation:** The fitted line is approximately:

$$\text{Salary} = 29826.21 + 8351.93 \times \text{YearsExperience}$$

Every additional year of experience is associated with roughly **\$8,351.93** more in predicted salary.

## 6. Model Evaluation

```

In [6]: y_pred_test = model.predict(X_test)

mae = mean_absolute_error(y_test, y_pred_test)
mse = mean_squared_error(y_test, y_pred_test)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred_test)

print(f"MAE : {mae:,.2f}")
print(f"MSE : {mse:,.2f}")
print(f"RMSE : {rmse:,.2f}")
print(f"R2 : {r2:.4f}")

```

MAE : 6,984.68  
MSE : 76,575,772.12  
RMSE : 8,750.76  
R2 : 0.9280

**Interpretation:**

- **RMSE  $\approx$  \$8,750.76** — on average, predictions are off by about this much in dollars.
- **$R^2 \approx 0.93$**  — the model explains about 93% of the variance in salary using years of experience alone.

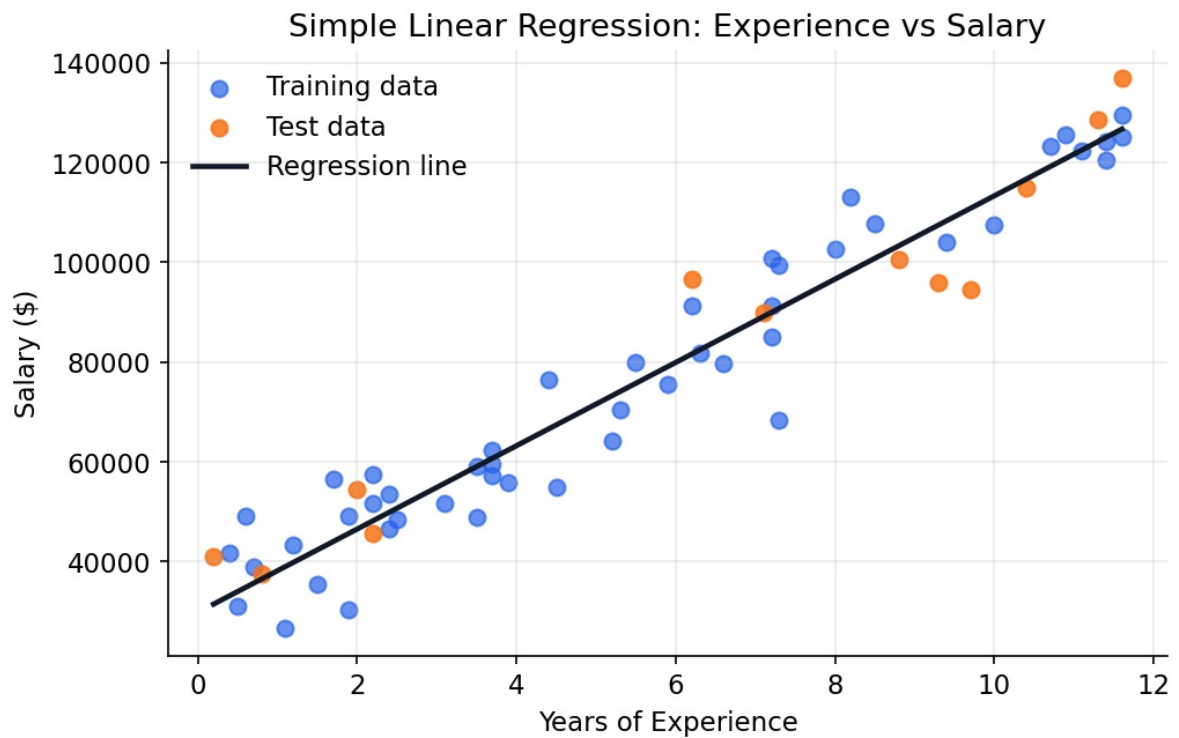
## 7. Visualizing the Regression Line

```

In [7]: x_line = np.linspace(X.min(), X.max(), 100).reshape(-1, 1)

plt.scatter(X_train, y_train, color="#2563eb", alpha=0.7, label="Training data")
plt.scatter(X_test, y_test, color="#f97316", alpha=0.85, label="Test data")
plt.plot(x_line, model.predict(x_line), color="#111827", linewidth=2, label="Regression line")
plt.xlabel("Years of Experience")
plt.ylabel("Salary ($)")
plt.title("Simple Linear Regression: Experience vs Salary")
plt.legend(frameon=False)
plt.grid(alpha=0.25)
plt.show()

```

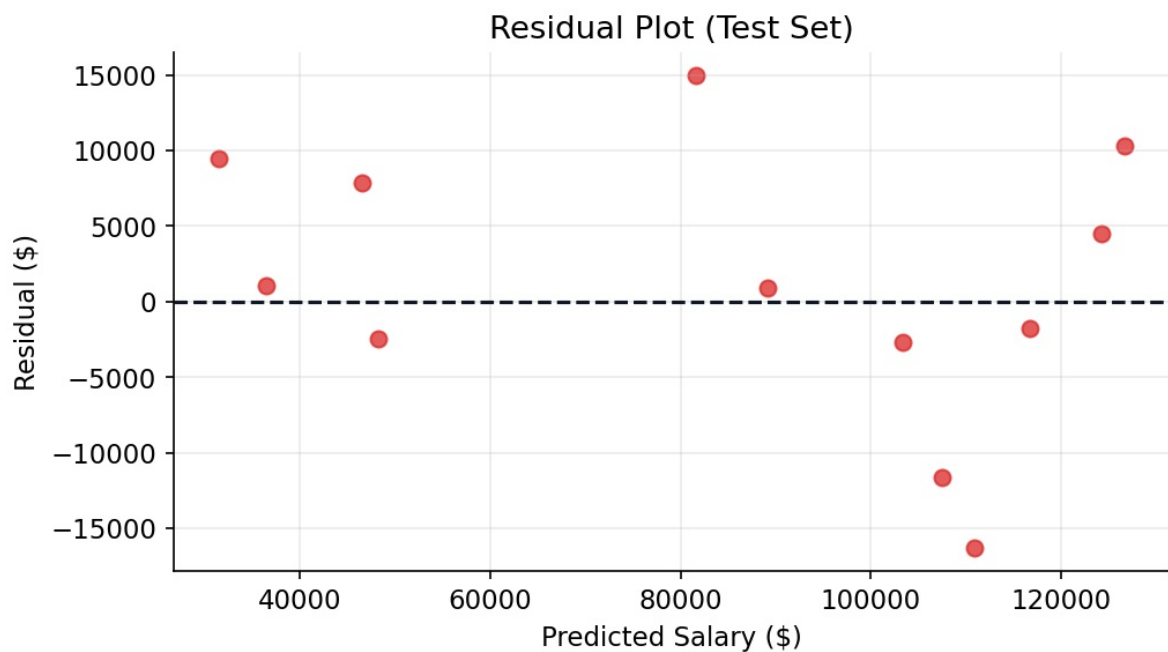


## 8. Residual Analysis

Residuals (actual – predicted) should scatter randomly around zero with no obvious pattern — this indicates the linear assumption is reasonable.

```
In [8]: residuals = y_test - y_pred_test

plt.scatter(y_pred_test, residuals, color="#dc2626", alpha=0.75)
plt.axhline(0, color="#11827", linestyle="--", linewidth=1.5)
plt.xlabel("Predicted Salary ($)")
plt.ylabel("Residual ($)")
plt.title("Residual Plot (Test Set)")
plt.grid(alpha=0.25)
plt.show()
```



## 9. Predicting on New Data

```
In [9]: new_years = [[5.0]]
predicted_salary = model.predict(new_years)
print(f"Predicted salary for 5.0 years of experience: ${predicted_salary[0]:,.2f}")
```

Predicted salary for 5.0 years of experience: \$71,585.86

## 10. Summary

| Metric               | Value     |
|----------------------|-----------|
| Intercept (b0)       | 29,826.21 |
| Coefficient (b1)     | 8,351.93  |
| MAE                  | 6,984.68  |
| RMSE                 | 8,750.76  |
| R <sup>2</sup> Score | 0.9280    |

### Key takeaways:

- Simple linear regression fits a straight line by minimizing squared errors (OLS).
- R<sup>2</sup> of 0.93 shows a strong linear relationship in this synthetic dataset.
- Always validate on a **held-out test set** — never evaluate only on training data.
- Real-world data is noisier; residual plots help diagnose whether linear regression is even the right model.

### Next steps

- Try **Multiple Linear Regression** with more features
- Explore **Polynomial Regression** for non-linear trends
- Add **regularization** (Ridge / Lasso) to control overfitting

---

★ If this notebook helped you, consider starring the repo on [GitHub](#).